



BITS Pilani
K K Birla Goa Campus

ADDRESSING MODES

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani Prabhu
Department of CS & IS



Register Addressing

- Transfers a copy of a byte or word from the source register or contents of a memory location to the destination register or memory location

e.g. MOV CX, DX

- copies the word-sized contents of register DX into register CX
- CX= 2A84H DX=4971H
- After instruction execution, DX= 4971H CX=4971H

Immediate Addressing

- Transfers the source, an immediate byte, word, doubleword, or quadword of data, into the destination register or memory location.

e.g. `MOV AL, 22H`

- Copies a byte-sized 22H into register AL.

e.g. `MOV CX, 437BH`

- Copied in CH and CL memory location

e.g. `MOV EBX, 12345678H` [In 80386]

- copies a doubleword-sized 12345678H into the 32-bit-wide EBX register

Direct Addressing

- Moves a byte or word between a memory location addressing and a register.
- The instruction set does not support a memory-to memory transfer, except with the MOVS instruction.

e.g. `MOV CX, LIST`

- copies the word-sized contents of memory location LIST into register CX

e.g. `MOV BL, [437AH]`

BIU calculates physical address

Consider DS=2000

Register Indirect Addressing

- Transfers a byte or word between a register and a memory location addressed by an index or base register.
- The index and base registers are BP, BX, DI and SI.

e.g. `MOV AX, [BX]`

`BX=1000H & DS=0100H`

- copies the word-sized data from the data segment offset address indexed by BX into register AX.

Base-plus-index Addressing

- Transfers a byte or word between a register and the memory location addressed by a base register (**BP or BX**) plus an index register (**DI or SI**).

The base register often holds **the beginning location of a memory array**, whereas the index register holds **the relative position of an element** in the array.

e.g. `MOV [ BX+DI ], CL`

- copies the byte-sized contents of register CL into the data segment memory location addressed by BX plus DI.)

Base-plus-index Addressing

Whenever BP addresses memory data, both the stack segment register and BP generate the effective address

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV CX,[BX+DI]	16 bits	Copies the word contents of the data segment memory location addressed by BX plus DI into CX
MOV CH,[BP+SI]	8 bits	Copies the byte contents of the stack segment memory location addressed by BP plus SI into CH
MOV [BX+SI],SP	16 bits	Copies SP into the data segment memory location addressed by BX plus SI
MOV [BP+DI],AH	8 bits	Copies AH into the stack segment memory location addressed by BP plus DI

Register relative Addressing

- **Register relative addressing** is a memory addressing mode where the **effective address (EA)** of the operand is obtained by adding a **constant displacement** to the content of a **base or index register**.

e.g.

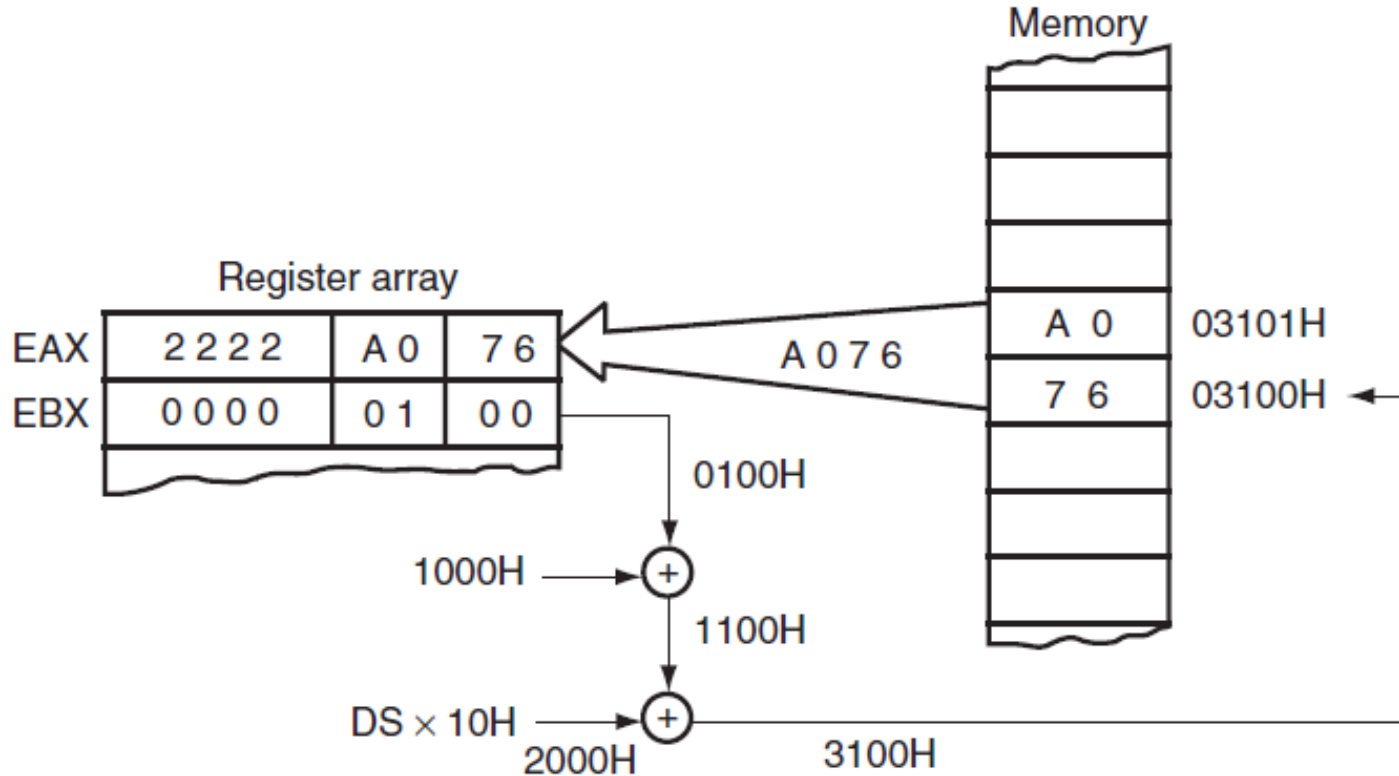
MOV AX, [BX+4]

- Loads AX from the data segment address formed by BX plus 4.

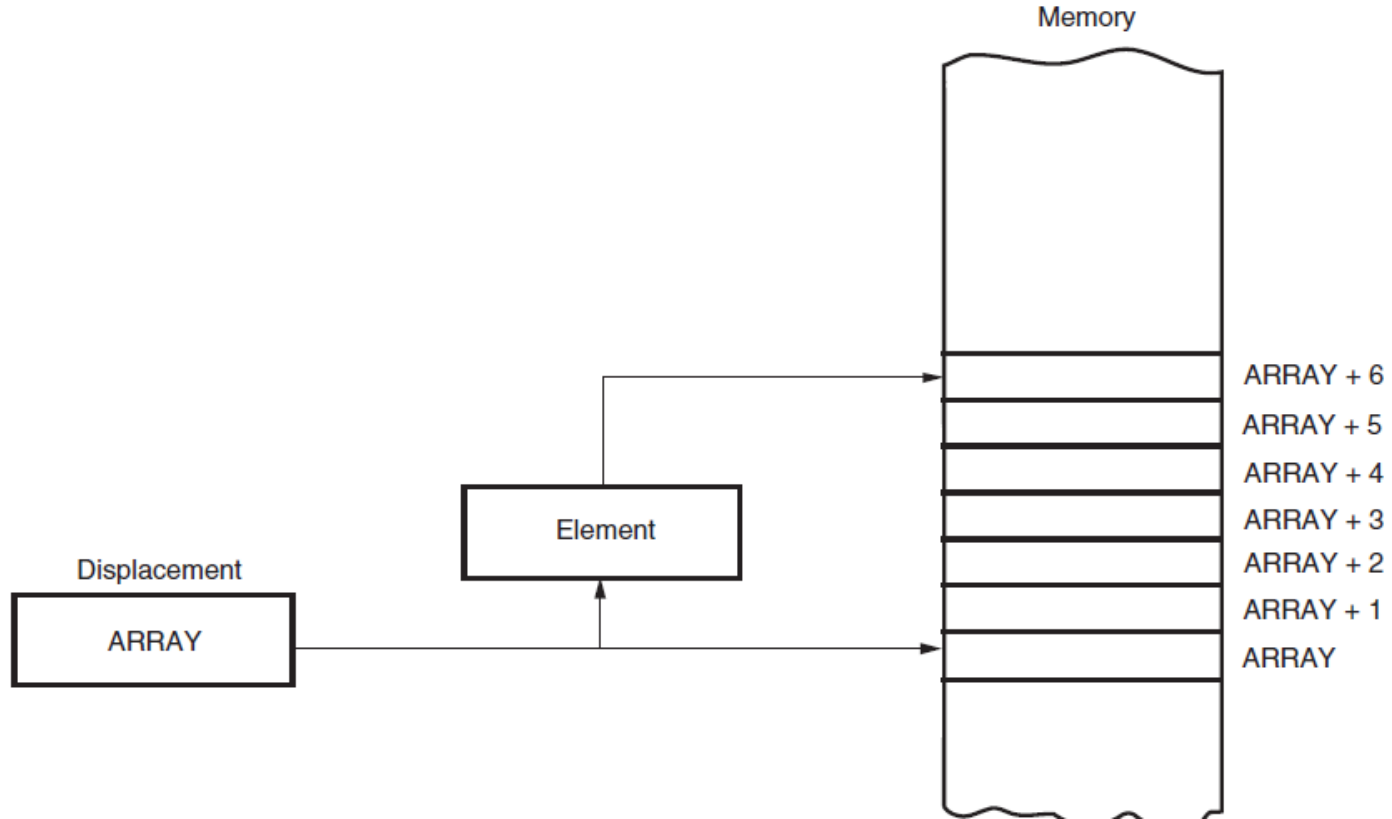
MOV AX, ARRAY[BX]

Loads AX from the data segment memory location in ARRAY plus the contents of BX.

Register relative Addressing: `mov AX,[BX+1000H]`



Register relative Addressing: Array



Register relative Addressing

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV AX,[DI+100H]	16 bits	Copies the word contents of the data segment memory location addressed by DI plus 100H into AX
MOV ARRAY[SI],BL	8 bits	Copies BL into the data segment memory location addressed by ARRAY plus SI
MOV LIST[SI+2],CL	8 bits	Copies CL into the data segment memory location addressed by the sum of LIST, SI, and 2
MOV DI,SET_IT[BX]	16 bits	Copies the word contents of the data segment memory location addressed by SET_IT plus BX into DI

Base relative-plus-index Addressing

- Transfers a byte or word between a index addressing register and the memory location addressed by a base and an index register plus a displacement.
- **Base register:** BX or BP
- **Index register:** SI or DI
- **Displacement:** 8-bit or 16-bit constant

e.g. `MOV AX, ARRAY[ BX+DI]` or `MOV AX, [BX+DI+4 ]`

- These instructions load AX from a data segment memory location.
- The first instruction uses an address formed by adding ARRAY, BX, and DI and the second by adding BX, DI, and 4.

Segment used

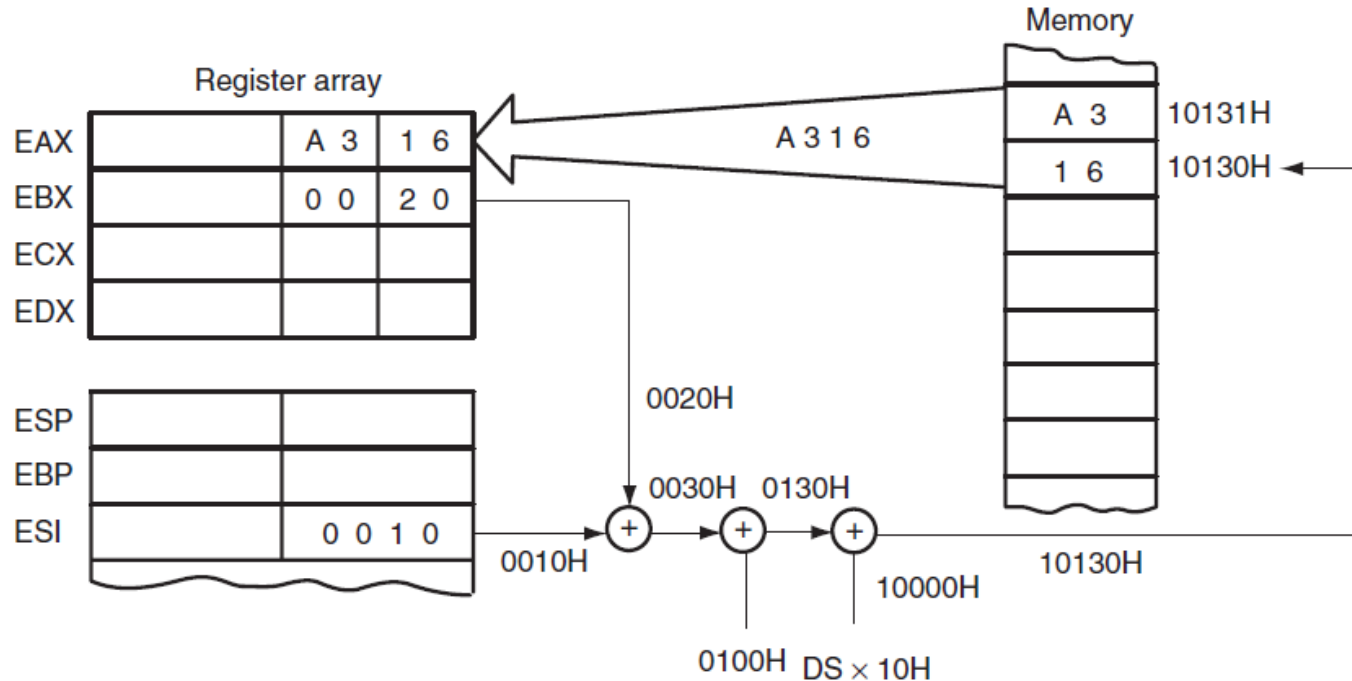
If **BX** $\rightarrow$ Default segment = **DS**

If **BP** $\rightarrow$ Default segment = **SS**

How it works

- Start from a **base address** (BX or BP)
- Move using an **index** (SI or DI)
- Add a **fixed offset** (8-bit or 16-bit constant)

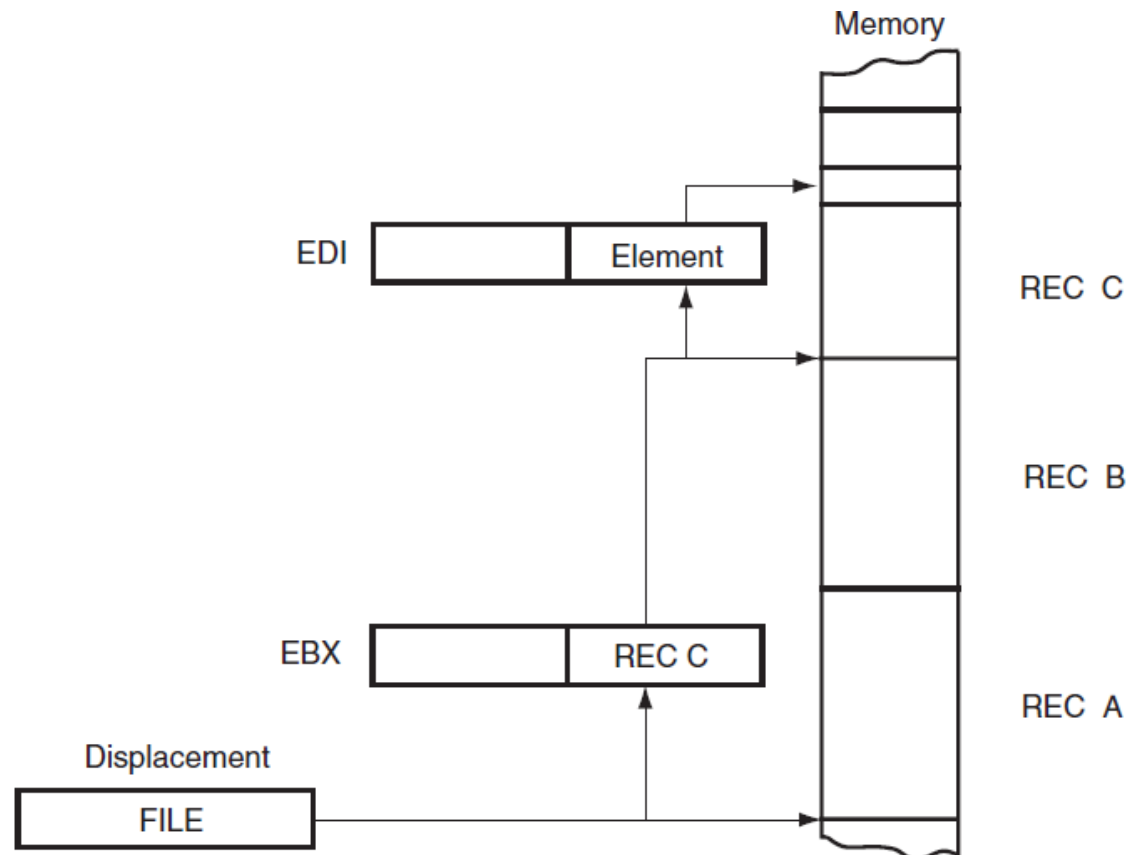
Base relative-plus-index Addressing: `mov AX,[BX+SI+100H]`



Base relative-plus-index Addressing: Accessing file with multiple records



BITS Pilani
K K Birla Goa Campus



Base relative-plus-index Addressing

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV DH,[BX+DI+20H]	8 bits	Copies the byte contents of the data segment memory location addressed by the sum of BX, DI and 20H into DH
MOV AX,FILE[BX+DI]	16 bits	Copies the word contents of the data segment memory location addressed by the sum of FILE, BX and DI into AX
MOV LIST[BP+DI],CL	8 bits	Copies CL into the stack segment memory location addressed by the sum of LIST, BP, and DI
MOV LIST[BP+SI+4],DH	8 bits	Copies DH into the stack segment memory location addressed by the sum of LIST, BP, SI, and 4

Scaled-index Addressing

- Available only in the 80386 through the addressing Pentium 4 microprocessor.
- Uses two 32-bit registers (a base register and an index register) to access memory
- Second register is modified by a scale factor of $2\times$, $4\times$, $8\times$ to generate operand memory address.

e.g. `MOV EDX, [EAX+4*EBX]`

- loads EDX from the data segment memory location addressed by EAX plus four times EBX.
- Scaling allows access to word ($2\times$), doubleword ($4\times$), or quadword ($8\times$) memory array data.

RIP Relative Addressing

- Only available to **the 64-bit** extensions on the addressing Pentium 4 or Core2.
- Allows access to any location in the memory system by adding a 32-bit displacement to the 64-bit contents of the 64-bit instruction pointer.

e.g. RIP = 1000000000H and a 32-bit displacement is 300H, the location accessed is 1000000300H.

- The displacement is signed so data located within $\pm 2\text{G}$ from the instruction is accessible by this addressing mode.

Addressing Mode	Example	Effective Address Calculation	Typical Use
Immediate	<code>MOV AX, 5</code>	Operand is a constant embedded in the instruction.	Loading constant values into registers or memory.
Register	<code>MOV AX, BX</code>	Operands directly in CPU registers.	Fast data access, arithmetic, and logical operations.
Direct	<code>MOV AX, [1234h]</code>	Address specified directly in the instruction.	Accessing global/static variables.
Register Indirect	<code>MOV AX, [BX]</code>	Address stored in a register.	Pointer dereferencing, dynamic memory access.
Base + Index	<code>MOV AX, [BX + SI]</code>	Base register + Index register.	Accessing arrays or data structures.
Register Relative	<code>MOV AX, [BX + 4]</code>	Base register + Constant displacement.	Accessing structure fields or local variables.
Base Relative + Index	<code>MOV AX, [BX + SI + 4]</code>	Base register + Index register + Displacement.	Complex data structures like records.
Scaled Index	<code>MOV AX, [BX + SI * 4]</code>	Base register + (Index register × Scale).	Arrays with elements larger than 1 byte (e.g., <code>int</code>).
RIP-Relative (x86-64)	<code>MOV RAX, [RIP + offset]</code>	Instruction Pointer (RIP) + Displacement.	Position-independent code for shared libraries.

Write an assembly language program in 8086 to find **add** n eight bit numbers.

Input Data	➡	03	45	32	8A
Offset	➡	500	501	502	503

Write an assembly language program in 8086 to find **add** n eight bit numbers.

0500H = 03H

0501H = 45H

0502H = 32H

0503H = 8AH

Store result at **0600H**

Write an assembly language program in 8086 to find **add** n eight bit numbers.

```
MOV SI, 0500H      ; Point to first number
MOV AL, 00H        ; Clear accumulator

ADD AL, [SI]        ; AL = AL + 03H
MOV SI, 0501H
ADD AL, [SI]        ; AL = AL + 45H
MOV SI, 0502H
ADD AL, [SI]        ; AL = AL + 32H
MOV SI, 0503H
ADD AL, [SI]        ; AL = AL + 8AH

MOV [0600H], AL    ; Store result
```

Using Different Addressing Modes

```
MOV BX, 0500H
MOV SI, 0000H
MOV AL, 00H

ADD AL, [BX]

ADD AL, [BX+1]

ADD AL, [BX+SI+2]

MOV SI, 3
ADD AL, [BX+SI]

MOV [0600H], AL
```




BITS Pilani
K K Birla Goa Campus

THANK YOU

